

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

Equipe NexaTech

Relatório da Sprint 1

Este documento lista somente falhas e inconformidades encontradas na avaliação da Sprint 1, conforme a rubrica fornecida.

Itens da rubrica sem falha objetiva identificada foram omitidos.

Engenharia de Software II

Item avaliado	Falha encontrada
1.1 README principal	O README da raiz não contém instruções de execução da aplicação nem estrutura de pastas suficiente para reprodução do ambiente. Também aponta para ./documentacao/modelagem_bd.md, arquivo que não existe no repositório.
1.2 README das pastas principais	A pasta principal backend não possui README próprio. Assim, a responsabilidade e a forma de execução/configuração do backend não ficam documentadas no próprio módulo.
2.1 Backlog do Produto	O arquivo documentacao/requisitos.md lista requisitos e user stories, mas não apresenta prioridades nem critérios de aceite por história, itens exigidos pela rubrica.
2.2 Backlog da Sprint 1	O backlog da Sprint 1 não informa responsáveis, status das tarefas nem vínculo claro com as disciplinas. A coluna Disciplina aparece sem preenchimento e a tarefa T10 tem marcação de tabela inconsistente.
2.3 Definition of Done (DoD)	O DoD exige TypeScript sem any, arquitetura MVC/Clean, variáveis sensíveis via .env, JWT/RBAC, rotas protegidas, API documentada e validação via docker-compose. A entrega não cumpre esses itens: há any/implicit any, backend sem camadas de serviço/repositório, conexão hardcoded, sem autenticação JWT/RBAC implementada e rotas administrativas sem proteção.
3.2 Coerência entre documentação e	Há divergência entre documentação, banco e código: o README descreve backend com JWT/RBAC, mas o package.json não possui dependências de JWT/bcrypt e não há middleware de

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

implementação	autenticação; o README informa healthcheck /health, mas essa rota não existe; a documentação do banco descreve users/navigation_nodes/inquiries/interaction_logs, enquanto o Prisma/backend trabalham com Documento/documentos; o frontend usa menus hardcoded em src/data/menus.ts em vez de consumir a árvore do banco.
4.1 Estratégia de branches	Na inspeção local foram encontradas apenas main e origin/dev. Não foram identificadas branches de feature/fix que evidenciem fluxo mínimo de trabalho por tarefa.
4.2 Distribuição temporal dos commits	Há concentração relevante no fim da sprint: vários commits de frontend, rotas, downloads, ajustes e reversões ocorreram entre 30/04/2026 e 05/05/2026, véspera/data da entrega.
4.3 Participação dos membros	A participação por commits está desequilibrada: Gustavo-Zago 13, Luka Gomes 9, Gabriel Moura 3, Ronaldo de Avila 2 e Breno Augusto 1. Isso indica baixa distribuição de contribuição registrada no Git.

Desenvolvimento Web II

Item avaliado	Falha encontrada
2.1 Estrutura em camadas	O backend concentra acesso a banco, manipulação de arquivos, regras de navegação e respostas HTTP em UploadController.ts. Não há services nem repositories, reduzindo a separação de responsabilidades esperada.
2.2 Organização das rotas	Há rotas duplicadas em src/routes/index.ts: GET /documentos aparece duas vezes e GET /navegacao/categorias/:curso aparece duas vezes. Uma das declarações de categorias contém handler vazio. A função navegarChat existe no controller, mas não está registrada nas rotas.
3.1 Uso de variáveis de ambiente	O backend ignora variáveis de ambiente essenciais: src/server.ts fixa PORT = 3001, enquanto o docker-compose publica a porta 3000; src/lib/prisma.ts usa connectionString hardcoded para postgresql://postgres:1234@localhost:5433/fatec_chatbot, ignorando DATABASE_URL do Compose.

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

3.2 Exposição de dados sensíveis	Há credenciais e segredos hardcoded: senha postgres:1234 em src/lib/prisma.ts; POSTGRES_PASSWORD padrão secretaria_pass, JWT_SECRET padrão change-me-in-production e senha padrão do pgAdmin admin123 no docker-compose.yml. Não há .env.example para orientar substituição segura.
4.2 Organização lógica do frontend	O frontend não possui camada de services, hooks ou contexts para integração com API. O fluxo do chatbot está hardcoded em src/data/menus.ts e o componente Chatbot.tsx manipula navegação, histórico e satisfação localmente, sem persistência ou consulta ao backend.
4.3 Estrutura visual	A tela inicial usa dimensões e deslocamentos fixos relevantes, como h-[600px] e translate-y-36 na imagem do hero, além de layout central com largura máxima fixa. Isso cria risco objetivo de quebra/overflow em resoluções menores, sem evidência de validação responsiva.

Banco de Dados Relacional

Item avaliado	Falha encontrada
1.1 Conexão PostgreSQL	A conexão configurada no Prisma não aponta para o serviço postgres do Docker Compose nem usa DATABASE_URL. O backend tenta localhost:5433/fatec_chatbot com usuário postgres, enquanto o Compose define postgres:5432/secretaria_db com secretaria_user.
1.2 Persistência correta	Dados relevantes do chatbot estão armazenados no frontend em src/data/menus.ts. O formulário “Envie sua Dúvida” não possui handler de envio e a avaliação de satisfação do chatbot fica apenas em estado React; não há persistência dessas interações no banco.
2.1 Qualidade dos comandos SQL	Há inconsistência entre DDL/DML, Prisma e backend: 01_schema.sql cria navigation_nodes/inquiries/interaction_logs, mas o Prisma define apenas Documento; o controller usa prisma.documentos, que não corresponde ao model Documento gerado pelo Prisma; fatec_chatbot.sql cria uma tabela documentos separada e termina com SELECT * FROM documentos, misturando script de consulta com script de inicialização.
2.1 Qualidade	O seed 02_seed.sql usa o mesmo slug dsm-portfolio para “Portfólio” e

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

dos comandos SQL	“Trabalho de Graduação (TG/TCC)”. Como a função faz upsert por slug, um item sobrescreve o outro e a árvore de navegação perde distinção entre opções.
2.2 Otimização inicial	O modelo SQL não mantém documentos oficiais, chunks e metadados em tabelas próprias, apesar de esses conceitos serem requisito do desafio. As evidências ficam como texto/link em navigation_nodes ou como arquivos/tabela documentos paralela, sem relacionamento claro com chunks indexados.

Técnicas de Programação I

Item avaliado	Falha encontrada
1.1 Aplicação funcional	Não foi possível comprovar build funcional. npm run build no backend falhou por ausência de dependências instaladas e erros TypeScript, incluindo implicit any em UploadController.ts e routes/index.ts. npm run build no frontend falhou com TS2688 por não encontrar type definitions vite/client e node.
1.2 Execução containerizada	O docker-compose define backend na porta 3000, mas o servidor Express escuta porta fixa 3001. Com isso, mesmo que o container suba, a porta publicada pelo Compose não corresponde à porta real da aplicação backend.
2.1 Tipagem TypeScript	Há uso de any explícito em UploadController.ts na leitura de MENSAGENS_FIXAS e erros de implicit any em callbacks e handlers. Isso viola o DoD de código 100% tipado e reduz a robustez da entrega.
2.2 Uso de conceitos de POO	Não há uso relevante de classes, encapsulamento ou abstrações orientadas a objeto. A lógica está em funções, componentes React e controller procedural, atendendo parcialmente ao estilo das tecnologias, mas fraco para o critério específico de POO.